n = 83 stacked producer with native work counted: section 5a band inputs

2026-10-23

n = 83 stacked producer with native work counted

Status (2026-10-23): counting run archived, v2 analysis `counts_identical`, band recorded for the stacked row. The frozen counting run (PROTOCOL.md, GitHub Actions run 39882571652, head `1a6067f6` on `claude/n83-opcounts-run`, AMD EPYC 9V45, rustc 1.99.0) is archived in `run_github_39882571652/`, and the complete-panel analyzer `analyze_v2.py` accepts it: all 62 rows in frozen order, every run valid, every output identical to the archived twin, one scalar per point. The band is in Results below. It is a band for the **stacked producer row**, not for the current rank-finder baseline row. Sections from "Which producer" to "The band script" describe the producer and the local diagnostics from the cloud-agent VM (4 vCPU Intel Xeon guest, rustc 1.97.0).

Question

AGENTS.md (crypto, section 5a) prices every arm's unpriced native work at 0, 0.1 and 1 group-addition equivalents (GAE) per operation and ranks two arms only where the order holds across that band. `docs/ic/baselines.json` kept the n = 83 band open because the frozen IC producer emitted no group-operation counts. This experiment adds the counts in a new producer version and the script that computes the band from them.

Which producer

The n83-stacked and n83-replay workflows build `experiments/koblitz-n83-stacked-20261016/online` (binary `ic_online_stacked_r770`): crypto `770f6a69` restored by that experiment's `restore.py`, plus the derived source `snapshots/r770/examples/koblitz_orbit_dlp_fast_online_stacked.rs` (sha256 `83b58988...`, pinned in its `protocol.json`), run one point per process by its `arm.py` through `run_schedule.one()`. The n = 83 path is the source's `mod wide` (`u128` words, 64 < n <= 127). The tuned rho `rho_tuned_r770` (`n83_rho_optimization_eval.rs`) already emits its charges (`charges_setup_and_solve`).

That source is frozen (its digest is pinned by the stacked and replay protocols), so nothing here edits it. `make_counted_source.py` regenerates it, checks the digest, and writes a new file, `derived/koblitz_orbit_dlp_fast_online_stacked_counted.rs` (sha256 `f95c5c10...`), built by `online/Cargo.toml` as `ic_online_stacked_counted_r770`. Counting is the Cargo feature `op-counts`; without it every counter compiles out (the control build).

What is counted

Per phase in the summary (`op_counts.phases`: setup, rank stage, linear algebra, targets, and the index digest after the targets, outside every interval) and per target in each target record (`op_counts`):

class	kinds
group	`ec_add` (generic affine add: 1 inv, 2 mul, 1 sqr), `ec_double` (1 inv, 2 mul, 2 sqr), `ec_add_trivial` (infinity operand or P + (-P)), `ec_scalar_mul` (a count; its steps are in the first three)
endomorphism	`frobenius_x` (x raised to 2^s, s != 0: scan rotations, shifted rows), `fold_rotations` (folded mates), `ec_frobenius` (base construction)
native (IC, not group)	`s3_root_solves`, `s3_root_solves_library`, `canonicalizations`, `root_index_probes`, `root_index_inserts`, `root_filter_queries`, `root_filter_inserts`, `relation_lifts`, `ec_lift_x`, `la_row_eliminations`, `la_mulmods`, `la_inversions`, `descent_mulmods`

class	kinds
field	`field_mul`, `field_sqr` (incl. linear-map squarings), `field_inv` (one per Itoh-Tsujii or library inversion), `field_half_trace`

The library group law is counted by wrappers that classify the operands with the library's own tests and then call `FastBinaryCurve128::add` / `double` unchanged. Scalar multiplication is the library's double-and-add loop over the same calls. Counters are per-thread `Cell`s (the producer is single-threaded), read only at phase boundaries outside the timed intervals.

Bit identity (local)

- `cargo test --release --locked --bins`, with and without `op-counts`: all 15 tests pass (the 13 stacked tests plus `counted_scalar_mul_matches_the_library` and `counting_leaves_relations_and_probes_unchanged`; the second hashes the relations, probes, root calls, skips and filter passes of five arms on 24 planted targets and must equal the digest `43045c80...` pinned from the uncounted build).
- `check_identity.py` (smoke/identity_local.json): the frozen binary, the uncounted build and the counted build, through the stacked `arm.py`, on Q0 (arm cS, 3 rotated rounds): identical relation, x-codes, intermediates, probes (1,987,852), root calls, relation checks, skipped states, scalar and index digests. kS at rank seed 101 (counted and uncounted) reproduces the archived K1 record (run 38862448935 serial 42): every attempt's probes, `rank_relations_blake3`, 1,000,454,051 rank probes, root calls and B's factor-log table. **All 11 checks pass.**
- `opcounts.py` smoke (smoke/opcounts_2points): kS, cS and rT on s000 and s001, `counts_identical`: every output equal to the archived run, the rho's charges included.

Overhead (local, diagnostic only)

Counted over uncounted, producer clocks, one pinned CPU on a shared VM: cold (setup + online) median 1.004 over 3 rounds, setup 1.005, index build 1.006, online 0.995; rank stage 1.023 (one pair: 313.4 s against 306.2 s). The uncounted build against the frozen binary reads 0.976 cold, so the VM noise is a few percent: the overhead is at most about 2% and not resolved below that here. The counting run records wall time but this experiment does not read it.

The band script

`band.py` reads `counts.json` (schema `ic-band-input/v1`): per arm, one unit per solved target. For IC, rank stage (from the K1 rank process) + setup + linear algebra + online (from the cold process at that point), which is the stacked `E2E(k, c)`. For rho, setup + solve. Priced cost = group ops + w x unpriced native ops, w in {0, 0.1, 1}, with one kind map for every arm (`CLASSES`). The rho's undecomposed scalar multiplications are bounded at 0 and 104 GAE each. Gaps to rT carry a paired bootstrap interval over points. A pair is ranked only if the order is the same at every weight and bound and every interval excludes 1; otherwise it is reported unresolved.

Local smoke band (2 points, S = kS + cS against rT; diagnostic, not the n = 83 result):

arm	group ops	unpriced native ops	cost w=0	w=0.1	w=1
S	1.011e5	3.093e9	1.011e5	3.094e8	3.093e9
rT	9.28e6	1.87e7	9.28e6	1.115e7	2.798e7

Gap S / rT: 0.0109 at w = 0, 27.7 at 0.1, 110.5 at 1, so on these two points the order flips inside the band: **unresolved**. With Frobenius applications priced as group operations (sensitivity), rT is cheaper at every weight.

Where the IC work is (kS, local): the rank stage makes 500,277,354 S3 root solves, 1,000,554,708 canonicalizations, 1,000,454,051 root-index probes and 494,249,916 Frobenius rotations, against 24,874 additions and 30,444 doublings. Setup makes 14.94 M root solves and 29.88 M inserts. One online target (Q0) makes 994,008 root solves and 1,987,852 probes, and 45 additions and 51 doublings.

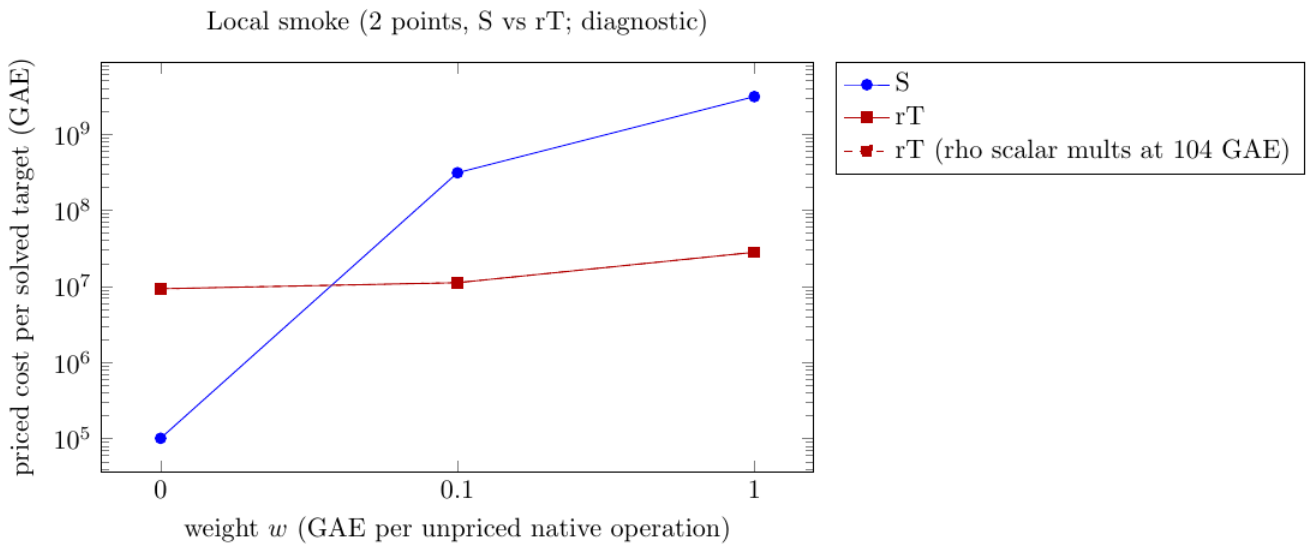


Figure 1: Local smoke band (diagnostic). Source: figures/smoke_band.tex.

Results: run 39882571652, v2 analysis

Evidence. `run_github_39882571652/` is the workflow artifact `n83-opcounts-39882571652-1` (id 12472682483, 396,291 bytes, zip sha256 `b1e43f37...3554`, equal to GitHub's digest), unzipped byte for byte; `ARTIFACT.json` records its provenance and `SHA256SUMS` the digest of every file. In the run, the workflow at `1a6067f6` gated with the frozen v1 analyzer (`opcounts.py analyze`, `analyze-stdout.txt`), which can accept a truncated panel (ANALYSIS_V2.md). The saved receipts were therefore re-analysed after download with `analyze_v2.py` (sha256 `42635193...8493`, checked against `ANALYSIS_V2.sha256`):

```
python3 experiments/koblitz-n83-opcounts-20261023/analyze_v2.py \
  --receipt-dir <copy of run_github_39882571652/panel>
```

Its outputs are in `run_github_39882571652/analysis_v2/` (`band-v2.md`, `counts-v2.json`, `opcounts_result-v2.json`, stdout). A second run reproduced every output byte for byte.

v2 verdict: `counts_identical` (exit 0, no mismatches):

check	result
`complete_frozen_panel` (62 rows, frozen serial, arm, point, seed, twin, block)	pass
`all_runs_valid`	pass
`identical_to_archived_run` (run 38862448935)	pass
`one_scalar_per_point`	pass

v2's band is identical to the v1 `band.md` from the run (and `counts-v2.json` to `counts.json`): v1 did not differ on this complete panel.

Band (20 points, reference rT; units: GAE per solved target; S = kS + cS, P = kP + cP, the stacked end-to-end cold `E2E(k, c)`; gap = cost(arm) / cost(rT), paired bootstrap 95% interval, 10,000 draws). Each gap cell is given with rho's undecomposed scalar multiplications priced at 0 GAE; at 104 GAE every cell moves by less than 0.1% (both ends in `band-v2.md`).

arm	mean group ops	mean unpriced native ops	gap w = 0	gap w = 0.1	gap w = 1
S	1.011e5	3.093e9	0.01138 [0.008767, 0.01552]	29.00 [22.33, 39.54]	115.5 [88.94, 157.5]
P	1.014e5	4.688e9	0.01141 [0.00879, 0.01556]	43.93 [33.83, 59.92]	175.0 [134.8, 238.7]
rT	8.882e6	1.79e7	1	1	1

Pair rulings (ranked only where the order holds at every weight and both rho bounds):

- S vs P: **S is cheaper than P at every weight** (S / P 0.9973 at w = 0, 0.6600 at 0.1, 0.6599 at 1).
- S vs rT: **unresolved**: S is about 88x cheaper at w = 0 and 29x to 116x dearer at w = 0.1 and 1; the order flips inside the band.
- P vs rT: **unresolved**, the same flip.

Sensitivity (reported beside the band, never instead of it): with Frobenius applications priced at 1 GAE as group operations, rT is cheaper than S and than P at every weight (S / rT 56.08 [43.18, 76.48], 70.99, 115.5; P / rT 84.00 [64.69, 114.6], 106.8, 175.0 at w = 0, 0.1, 1).

The distance remaining to rho for the stacked row is thus a factor of about 29 at w = 0.1 and 116 at w = 1, carried by the rank stage's ~3.1e9 native operations (root solves, canonicalizations, root-index probes and Frobenius rotations) against ~1.0e5 group operations. This is accounting of an existing producer, not an improvement, and no baseline row changes.

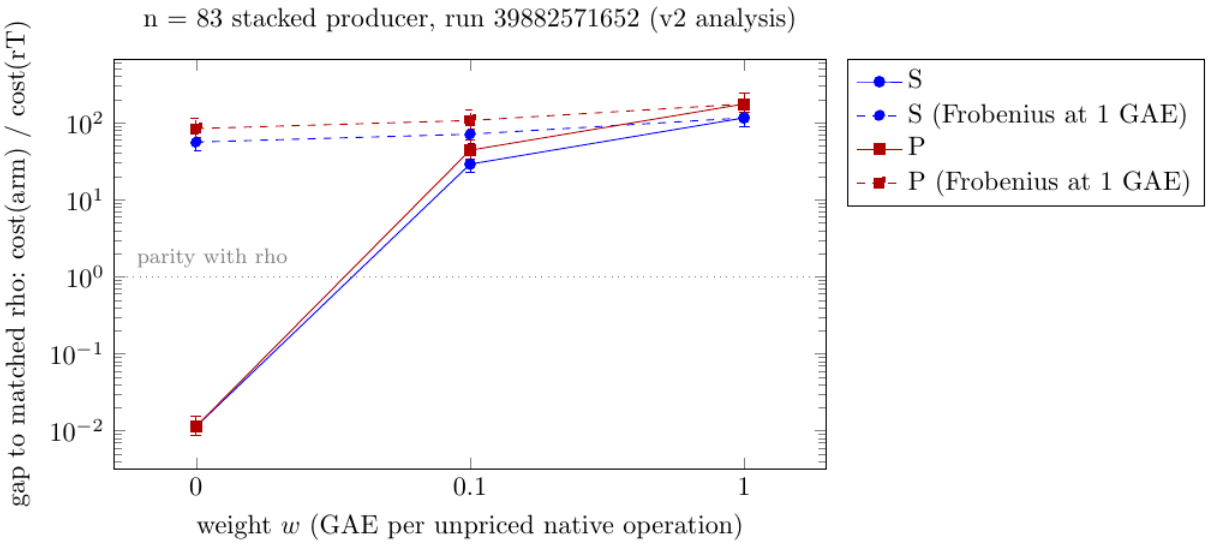


Figure 2: Gap to matched rho against the GAE weight, run 39882571652, v2 analysis (log axis; solid: band, dashed: Frobenius-at-1-GAE sensitivity; bars: 95% intervals). Source: figures/band_gap_39882571652.tex, SVG, drawn by `make_gap_figure.py`.

What is left

1. Done (2026-10-23): run 39882571652 archived in `run_github_39882571652/`, v2 verdict `counts_identical`, band above (stacked row only).
2. The current n = 83 baseline row is the rank-finder producer (`koblitz-n83-rankfinder-20261021`, derived from this source through the later index-lift, closure, index-build and rank-finder edits). Its band needs the same counters on that derivation (the edits here are anchored on text that later derivations may have changed) and its own counting run.
3. The rho's scalar multiplications stay a bound until its producer decomposes them.

Files

`make_counted_source.py` (derivation), `online/` (crate), `band.py`, `opcounts.py` (frozen driver), `check_identity.py`, `make_figure.py`, `PROTOCOL.md` and `PINS.sha256` (frozen protocol), `analyze_v2.py` and `ANALYSIS_V2.md` (complete-panel analysis), `make_gap_figure.py` (gap figure), `run_github_39882571652/` (the run and its v2 analysis), `smoke/` (local diagnostics), `figures/`.